Mobile Application Design 2025 project

VQueue

team:

• **Adam Selmane**, xselmaa00

• **Eyobed A. Nuri**, xnuriey00

• **Kaung San**, xsankau00

• **Krystof Michalek**, xmicha94

• **Lucas Labhini**, xlabhil00

Project Goal

- Provide a convenient alternative to physical queues
- Allow people to line up without standing in a physical line
- A user creates and manages a queue using their phone
- Other users join the queue using the same app
- The app notifies users when it is their turn
- Offline, Uses Bluetooth Low Energy (BLE), a range of approximately 100 meters
- no sign up/sign in

Used Technologies

- Java and XML in Android studio with Physical devices used as a test device.
- Bluetooth LE API : for wireless fuctionalities described above.
- Room DB: Store states.
- Material design 2.0

All neccessry requirements of the app are available in android SDK so nothing extra is needed to build or run.

App supports Android versions 5.0 to 15.

Used Resources

Code

- RecyclerView
- DiffUtil
- Templates from developer.android.com

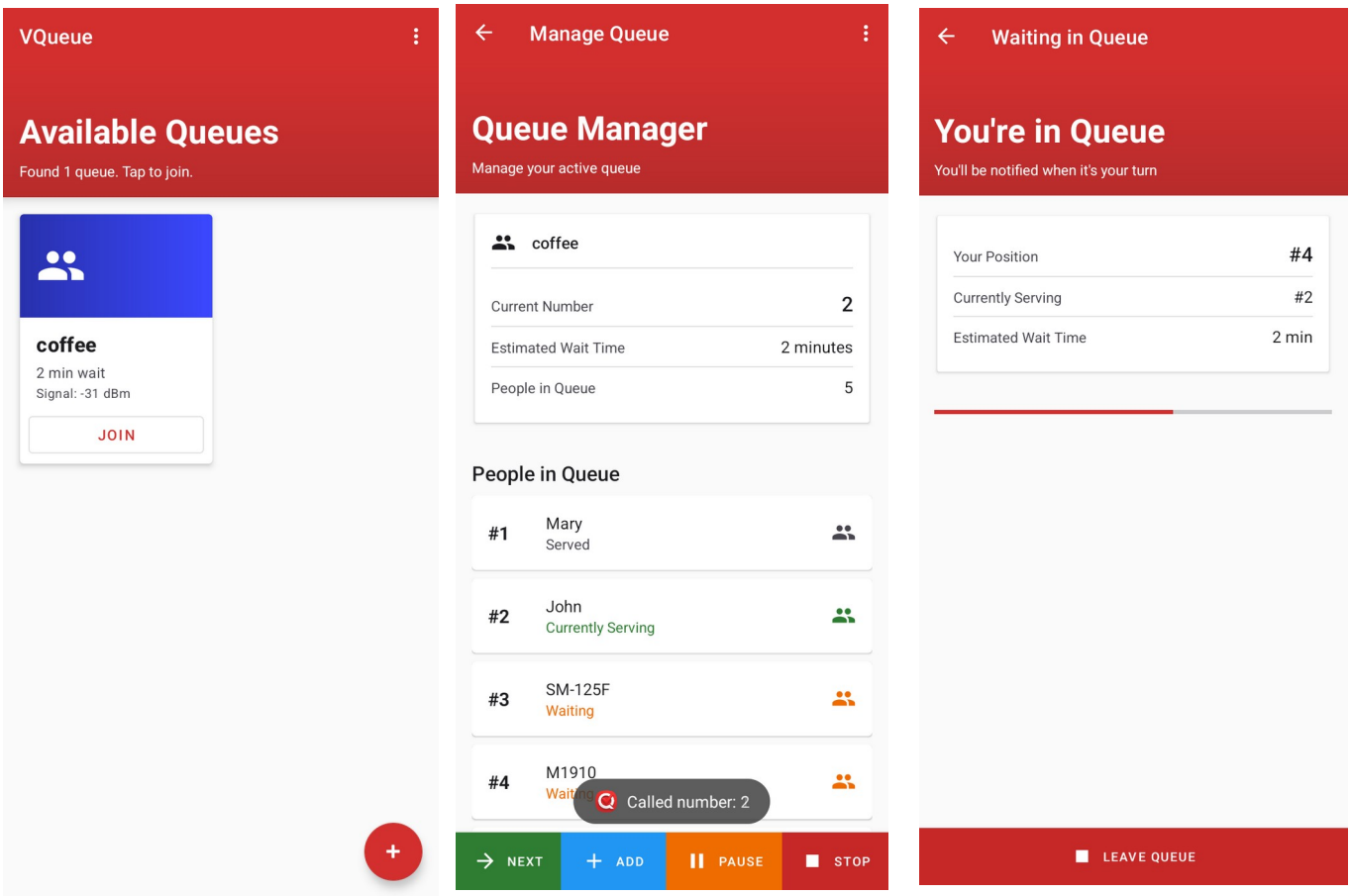
existing data

- Android studio built in clip arts

Study materials & Documentation

- Android's Docs at developer.android.com
- Material Guidelines at m2.material.io

Most Important Achieved Results



we added 3 screen shots which is essentially all of the app to demonstrate. In order :

1. shown on all devices on app start up. It scans the area for BLE signals specific to this app. Here we have a `coffee` queue for instance.

2.Is the screen for the `coffee` provider, which they created using the + FAB. They have all the queue for the `coffee` and a small panel to manage it

3.Is the screen for those in the line. With some info and a progress bar updated in real time

whats Important result here:

1.Custom P2P BLE protocol, real time and offline, which is harder to develop than web based centralized online solution

2.Material design guidelines, followed to our max ability which we belive is elegant

3.wide range of device support android 5.0 – 15

Controlling of Created Program

In short :

1. Queue manager creates a queue
2. Customers scan nearby area, app lists the queue(s) and customers join their selection
3. Queue manager sees list of people joined, opertation is first come first served
4. Customers get notified when it's their turn

Some additional info :

Queue managers can

- manually add people (those with no phone or IPhones),
- pause the queue(prevents more people Joining)

once setup, both managers and customers can close the application and do something else with their phones.

Theming is automatic. Users can set their preference in settings.

Experience with the Selected Platform

We selected the native Android, used Java and targeted Android 15 to comply with Google play requirement. We chosen this for better low level hardware access specifically for Bluetooth LE but sill maintain compatibility across a wide range of devices.

What Pleased Us

Jetpack Libraries

- Navigation Component: This made Fragment transitions simple instead of writing it manually
- Room Database: Some DB was needed to store queue info so the list or the turn would not be wiped if the device killed the process. Room database simplified that.
- ViewModel: separate UI from logic to make data survive screen rotations

Material UI : Nice to follow guides on how to make something look great

Documentation : The official Android docs is vast and contains sample code and best approches

What Disappointed

Bluetooth LE problems

- Hard to debug and surprise limitations
- Fragmentation
- APIs : They always change. Almost everything is changed from SDK 21 to 35. especially Bluetooth API’s Maintaining a minSdk of 21 while targeting 35 requires constant if else checks.

UI Glitch on diffrent versions of androids we tested

Distribution of the Work in the Team

Eyobed : developing concept, Initial architecture, Created UI + regular updates, did queue logic, did BLE logic, Handled merge requests, Handled GooglePlay deployment , presentation and docs, Physical device testings, coordinated the team.

Lucas : developing concept, Improved UI, Fixed navigation bugs and errors, Fixed duplicate Instance Bug. presentation and docs

Adam : developing concept, Added useful features to BLE logic(Sort by RSSI : Translates to physical closeness), presentation and docs

Krystof : developing concept, presentation and docs,Executed project setup, Bug fixs

Kaung: developing concept, Participate in UI(some work for dark mode).Bugs fix, Physical device testings

What Was the Biggest Challenge

The app's UI is rather intentionally simple as compared to the BLE implementation that had to be non standard to support our goal. To make an effective Queue management the main device needs to support an unlimited number of client devices, but BLE typically limits a single device to ~7 concurrent connections.

To deal with this, we implemented an algorithm combining advertising and a temporary GATT server. Clients connect briefly to obtain a position, then immediately disconnect. Subsequent updates are broadcast via the Advertising API, so unlimited clients to listen passively without occupying a slot.

Experience Gained From the Project

We learnt about

- Activity Lifecycles, Special android uitlity libs like DiffUtil, Jetpack navigation, Room DB etc...
- Material 2.0: Guidelines and Accessible color pallete selection. Some experiments with 3.0
- Blutooth Low energy API and good knowledge of the standard in general
- How to deploy an app : Policies, versioning, closed tests etc...

What we have Discovered

Important to mention here are :

- **Hybrid BLE Architecture**: Solving the connection limit by combining transactional joining with passive broadcasting. Mentioned in detail before.
- **Custom Data exchange protocol**: byte packing to fit app identification,queue information, and current status into 19 bytes of tiny BLE packets. Without this solution, the app would simply not work.
- **Quirks of deployment** : Publishing an app to PlayStore is no longer trivial, problems with their solutions had to be discovered.

Autoevaluation

Technical Design: 80%

We judge the design elegant. We adopted standard MVC and separation of Logic and UI. Single activity fragment based approach which is modern way. The data is modeled in objects which is welcome. Not using the latest MVVM and No automated testing: -20%

Programming: 75%

All code is native so speed and stability are good even on slow devices. The BLE logic can be generalized to be used in similar applications. Communications are Insecure. So we give 75%

Usability of the Created Solution: 80%

zero internet & hardware requirement and no signup for both parties unlike online solutions. the Material 2.0 aesthetics are beautiful(subjective), Application is tiny(1.5MB) whether users are willing to download an app for a queue, remains to be seen...

Use of Resources: 85%

We spent some time reading android documentation as there are few if not no examples of similar applications. Particularly BLE. There are no external libraries in the project except ofcourse Material UI, Which we also consulted with its guides to properly implement it.

Time Management: 70%

Average. Project was completed not too fast to do any extra features or not too slow to have missing parts. Some rigorous tests on BLE across many kinds of devices would have been good for reliablilty. -30 for this. 70%

Team Cooperation: 50%

Not great. We used Git/Gitlab to collaborate and sync. There were also a few meetings in person and online.

Chances of Publishing the App: 95%

As recommended, The team contributed the store fees and completed all preliminaries. App is deployed but publication is delayed due to problems with testers dedication. Probably will go live by mid january,2026. -5% in case Google rejects it.

To verify: published for testers here https://groups.google.com/g/vqueue/ then PlayStore Link

Overall Impression: 80%

We believe the application we built is quite good enough to demonstrate proper completion of the course, TAMa. We consider Bluetooth enabled app and native development rather non trivial and a good exposure. Probably the app will see real world usefulness since we deployed it to the store

Recommendation for Assigning Future Projects

We appreciate sectioned deliveries and workshops where the team presents progress at the middle. Which would boost motivations and could potentially avoid last minute hurries. We recommend that version control to be mandatory part of the project and evaluations to be conducted mid semester on the half-done project instead of a final evaluation which In our opinion improves time management

Recommendation for Future Students

Since our first title was rejected we recommend starting early and put great empahsis on selecting a

good topic to work on as that has a significant out come on what the rest of the assignment would look like. Testing the application is also as important as programming it if you want actual users. We also suggest starting deployemnt ealy if the team decide to publish their apps as Googleplay is very unpredictable when it comes to schedules. We recommend working on Android or Harmony platform as the other alternative, IOS is not friendly to students.